Data Science Project Technical Report

Mikko Anttonen, Denis Davydov, Roope Kolehmainen,
Lasse Vilpponen, Jianrui Xie

May 17, 2025

Abstract. We propose a neural network (NN)-based model featuring a CNN encoder and a transformer-based decoder, which outperforms the client's random sampling and heuristic models. On the 4-qubit benchmark, our model synthesizes circuits with CNOT counts only 27% above the optimum. A hybrid model combining our NN-based with the client's heuristic yields further improvement, reducing the gap to just 17%. Additionally, our reinforcement learning (RL)-based model, using a Deep Q-Network (DQN) agent, surpasses the random sampling baseline but underperforms relative to the heuristic, with CNOT counts averaging 30–40% above the optimum (on circuits with limited complexity). We also experimented with CNN-LSTM architectures, though these remain under evaluation.

1 Introduction

Despite advances in quantum computing, current quantum computing architectures still impose limitations on qubit connectivity, necessitating oversized quantum circuits with additional gates to route interactions. Specifically, hardware constraints allow only certain qubits to be physically connected. When two non-adjacent qubits need to interact, extra SWAP gates—each typically involving 3 CNOT gates (Appendix A.2)—are required to bring them adjacent. Adapting a quantum circuit to a particular device's connectivity constraints is known as the qubit routing problem. Optimizing circuits for this problem is crucial because qubits decohere over time and thus lose information, making deeper circuits noisier and less reliable.

A classical approach to the qubit routing problem inserts SWAP gates indiscriminately whenever two qubits are not adjacent, increasing circuit size and depth (Appendix B). State-of-the-art strategies leverage architecture-aware synthesis techniques. Rather than swapping qubits, these methods consider the hardware architecture and synthesize a new circuit, enabling global circuit optimizations. For example, the client proposed two architecture-aware Clifford synthesis algorithms [4, 6]. Their methods iteratively select

a pivot qubit and apply Clifford gates to transform the tableau into an identity matrix. Pivot choices, either pivot qubit indices [4] or pivot index pairs (row, column) [6], are the crucial first step that theoretically influences the minimization of CNOT gates. However, two selection strategies the client proposed most recently [6]—one random sampling and one randomized greedy selection based on the cost approximation of the Steiner tree—did not yield asymptotic improvements in reducing CNOT gates or improving scalability.

The project is a part of the client's future work and explores the possibility of machine learning techniques in developing a more effective pivot selection strategy. The project aims to predict the optimal permutation for pivot row and pivot column indices at the smallest cost for any circuit. We compare our results with three models: the random sampling model ($\mathcal{M}_R$) [6], the heuristic (greedy) model ($\mathcal{M}_H$) [6], and the theoretical optimum. Measurement of the number of CNOT gates (denoted by C_{cx}) takes priority, and the circuit depth (denoted by C_{dep}) serves as an auxiliary metric.

2 Data Generation and Preprocessing

We introduce several definitions for the training data structures.

Definition 1. *The Clifford tableau of an n-qubit circuit C_n is a mathematical representation in $\{0,1\}^{2n\times 2n} \times \{0,1\}^{2n}$, defined by*

$$C_n \equiv \left\{ \begin{pmatrix} X_s & Z_s \\ X_d & Z_d \end{pmatrix}, V_{sign} \right\},$$

where X_s, Z_s, X_d, Z_d are matrices in $\{0,1\}^{2n\times 2n}$ and V_{sign} is a vector in $\{0,1\}^{2n}$. Particularly, X_s and X_d represent Pauli-X's impact on stabilizers (first n rows) and destabilizers (last n rows), while Z_s and Z_d represent Pauli-Z's impact on stabilizers (first n rows) and destabilizers (last n rows). V_{sign} represents signs of (de)stabilizers.

Definition 2. *Let S_n be the symmetric group on $S = \{1, 2, \ldots, n\}$. Define the sequence space of permutation tuples by*

$$\mathcal{P}_n \coloneqq \left\{ \left((\sigma(i), \tau(i)\right)_{i=1}^{n} \;\middle|\; \sigma, \tau \in S_n \right\}.$$

Definition 3. *Let $p_n = \left((\sigma(i), \tau(i))\right)_{i=1}^{n} \in \mathcal{P}_n$. Define its standard matrix form by*

$$p_n \equiv \{P_i\}_{i=1}^{n}, \quad P_i = I_n + (\mathbf{e}_{\sigma(i)} - \mathbf{e}_{\tau(i)})(\mathbf{e}_{\sigma(i)}^{\top} - \mathbf{e}_{\tau(i)}^{\top}),$$

where each $P_i \in \mathbb{R}^{n\times n}$ is the permutation matrix obtained by swapping rows $\sigma(i)$ and $\tau(i)$ of the identity matrix.

Definition 4. *Let $p_n = \left((\sigma(i), \tau(i))\right)_{i=1}^{n} \in \mathcal{P}_n$. Define its cumulative matrix form by*

$$p_n \equiv \{T_i\}_{i=1}^{n}, \quad T_i = P_i T_{i-1}, \quad T_1 = P_1,$$

where $\{P_i\}_{i=1}^{n}$ is the standard matrix form of p_n.

Let $\mathcal{P}_n^*$ be the set of all optimal pivot selections for a given n-qubit circuit C_n, then $\mathcal{P}_n^* \subset \mathcal{P}_n$. Training data are triples of $(C_n, \mathcal{P}_n^*, C_{\text{cx}}^*)$, where $\mathcal{P}_n^*$ are generated by brute force through $\mathcal{P}_n$ with the minimum C_{cx}^* measuring by circuit synthesis. Recall that the complexity of $\mathcal{P}_n$ falls under super-exponential time $((n\,!)^2)$. It empirically takes 40-45 hours to generate 3,200 pairs of $(C_5, \mathcal{P}_5^*)$. Therefore, we limited our investigation mostly to 4-qubit circuits. Note that this is a multi-target prediction task since the prediction is one member $\hat{p}_n$ of $\mathcal{P}_n^*$, not the whole $\mathcal{P}_n^*$.

Among training data, circuits C_n are represented by Clifford tableau and encoded by 2-channel tensors. Regarding target labels $\mathcal{P}_n^*$, only the model in Section 3.1 adopted the cumulative matrix form, while the model in Section 3.2.1 used a cost matrix $\mathcal{Y}_n^* \in \mathbb{R}^{n\times n}$ as substitute. Each entry $\mathcal{Y}_{ij}$ sums up the immediate CNOT cost from the pivot (i, j) and the future best CNOT cost $\tilde{C}_{\text{cx}}^*$, effectively an $(n-1)$-qubit optimization by brute force. It is trivial to derive $\mathcal{P}_n^*$ from $\mathcal{Y}_n^*$ but not the reverse.

3 Model Selection and Design

3.1 CNN encoder with transformer-based decoder

Recall that any member of $\mathcal{P}_n^*$ has a length of n. Hence, we considered the following two modes. Theoretical Mode only accepts single-valued-qubit training data as the predicted length is always n. Pragmatic Mode handles variable-qubit training data with a stop head for control, which is expected to loosen the complexity constraint in higher-qubit training data generation. Their differences mainly lie in the output layer (Section 3.1.1.3) and the loss function (Section 3.1.2). Unless specified, discussions are in Pragmatic Mode.

3.1.1 Architecture

3.1.1.1 Encoder. Three convolutional layers constitute the encoder with GELU activation for smoother gradients and padding for spatial dimension preservation. While normalization is applied after the first and second layers to stabilize training, a pooling layer follows the third, ensuring a consistent output size of 4×4 for arbitrary n. Progressive channel expansion is utilized across convolutional layers from 2 to 32, then to 64 channels.

Several variants of this single-pathway encoder were tested. For example, a tableau-aware single-pathway encoder was built by widening the third layer to 128 channels and the kernel from 3×3 to 5×5 to capture broader patterns and then combining the pooling layer with a linear projection layer to create positional embeddings. A more tableau-aware encoder was modified multiple-pathway. It separately extracted features from Pauli-X's and Pauli-Z's in the input tableau, which were then concatenated by a bottleneck layer and processed by additional convolutional layers with pooling. Table 1 shows that these extra complexities have scarce gains in performance.

We also considered a standard transformer-based encoder with 8 attention heads and

positional embeddings about rows and columns. Due to $\mathcal{O}(n^2)$ complexity from self-attention, the encoder was considerably more time-consuming with worse performance (Table 1) than CNN encoders. We further designed a hybrid encoder that integrates self-attention between convolutional layers to leverage the ability to capture long-range dependencies in transformers. The learnable parameter γ in the self-attention implementation was initialized as zero, allowing a smooth transition from the pure CNN encoder to the hybrid encoder. However, the hybrid encoder performed decently (Table 1).

3.1.1.2 Decoder.

A simple standard transformer-based decoder by calling

```
nn.TransformerDecoder(nn.TransformerDecoderLayer(…,activation=F.gelu))
```

worked surprisingly well. Meanwhile, we designed a CNOT-aware variant that introduces CNOT attention weights to link permutation choices with C_{cx} optimization. The CNOT attention weights are the projection of the decoder output to $\mathcal{P}_n$, later constrained by a sigmoid function and integrated with regular features to create CNOT-aware representations.

3.1.1.3 Output layer.

In Pragmatic Mode, there are three heads in the output layer: a position-specific step head that generates pivots in order, a stop head that predicts when to stop the step head, and a CNOT head that predicts C^*_{cx}. The autoregressive loop of the step head has a hardcoded maximum length (10). On the contrary, Theoretical Mode does not have a stop head since the loop follows n.

A simple CNOT head with 2 linear layers and ReLU activation already performed nicely (Table 1). However, we gradually enhanced it by first adding permutation features from the output of the step head, then adding CNOT awareness by CNOT attention weights from the decoder, then adding a dropout regularization to prevent overfitting and a residual connection to improve the gradient flow, and finally adopting GELU in hidden layers for smoother gradients but ReLU in the output layer for sharper predictions. Experiments implied that this sophisticated CNOT head barely improved performance.

3.1.2 Loss function

The loss function[1] in Pragmatic Mode is given by

$$\mathcal{L}(\hat{p}_n, \hat{C}_{\text{cx}}) = \operatorname*{avg}_{\mathcal{B}} \min_{p_n \in \mathcal{P}^*_n} \text{MSE}(\hat{p}_n, p_n) + \alpha \cdot \text{MSE}(\hat{C}^*_{\text{cx}}, C^*_{\text{cx}}) + \beta \cdot \mathcal{L}_{stop} + \gamma \cdot \|\,|\hat{p}_n| - n\|,$$

where BCE between predicted stop logits from the stop head and the stop labels measures the stop signal loss. Theoretical Mode has no stop signal loss or length regularization.

We considered a compound permutation loss that integrates KL divergence with multiplying logits by 10 before calculating MSE, hoping to obtain sharper $\hat{p}_n$. We attempted to encourage exploration by adding entropy loss, which averages entropies of probability distributions from normalizing each $\hat{T}_i \in \hat{p}_n$. However, none brought improvement.

[1] Measuring the CNOT loss by MSE is a mistake. See Section 3.1.5.

3.1.3 Training and inference

The training optimizer is fixed to `AdamW`, while the scheduler varies among experiments. Over time, cyclical learning rate policies slightly outperformed cosine annealing schedules with and without warm starts (Table 1).

The training and inference processes are mostly standard. For example, the Hungarian algorithm converts predicted permutation logits to a hard permutation; top-k candidates are chosen for evaluating the actual cost by circuit synthesis.

Novelty of Gumbel-Sinkhorn Sampling: During training, predicted permutation logits were passed through Sinkhorn normalization to obtain doubly stochastic matrices. At inference, Gumbel noise was added to the logits before Sinkhorn normalization, allowing sampling soft permutation matrices, which were then discretized. This distinction arises because the deterministic and differentiable Sinkhorn normalization supports stable gradient-based optimizations during training. In contrast, Gumbel-Sinkhorn sampling introduces stochasticity to encourage exploration in $\mathcal{P}_n$ with diversity.

Novelty of Probabilistic Weights[2]**:** Rather than sorting $\hat{T}_i$ deterministically by $\hat{C}^*_{\text{cx}}$, we introduced CNOT weights to bias the subset selection of $\hat{T}_i$ to favor lower $\hat{C}^*_{\text{cx}}$:

$$W_{\text{cx}} \coloneqq \exp(-2\hat{C}^*_{\text{cx}}), \quad p_{\text{cx}} = \text{Softmax}(W_{\text{cx}}).$$

We sampled by the probability distribution p_{cx} a subset of indices for $\hat{T}_i$. Compared with greedy selection, this approach smooths out noises and achieves a good balance between exploitation and exploration, and thus is more robust. We attempted to enhance the inference by entropy-guided search. CNOT weights were replaced by $\mathcal{H}(\mathbf{P}_i)$, where $\mathbf{P}_i$ are probability matrices generated from applying Sinkhorn normalization on logits $(\hat{T}_i)$. A subset of $\hat{T}_i$ with highest $\mathcal{H}(\mathbf{P}_i)$ was selected to continue with inference (i.e., Gumbel-Sinkhorn sampling, conversion by the Hungarian algorithm, evaluation by circuit synthesis). This approach enabled further exploration in areas the model is most uncertain about (Table 1).

Minor modifications include replacing the fixed temperature (0.1) in Gumbel-Sinkhorn sampling with a small range of values and a final ensemble strategy.

3.1.4 Performance

Table 1 illustrates experiment results on the model architecture, while Table 2 gives more promising performance from the mixed model combining our best NN-based model with the model $\mathcal{M}_H$. All performances were evaluated over independent sets of 1000 fresh random circuits, each with 1000 gates.

With CNN encoder, transformer-based decoder, Gumbel-Sinkhorn sampling in inference, and entropy-guided search, our best NN-based model won a landslide victory over the model $\mathcal{M}_R$ in both C_{cx} (-3 gates) and C_{dep} (-4). Further, it outperformed the model $\mathcal{M}_H$ significantly in C_{dep} (-2) and weakly but steadily in C_{cx} (-0.2 gates). Though the mixed model introduced a remarkable improvement in both C_{cx} and C_{dep}, a noticeable

[2]Treating $\hat{C}^*_{\text{cx}}$ as a sequence and $\hat{T}_i$ as the cumulative effect of p_n are mistakes. See Section 3.1.5.

gap in C_{cx} (+1 gate in the 4-qubit benchmark, +2 gates in the 5-qubit benchmark) from the optimum exists.

Model variant	ΔC_{cx} [1]			ΔC_{dep} [2]		
	$\mathcal{M}_{NN}$ [3]	$\mathcal{M}_H$	$\mathcal{M}_R$	$\mathcal{M}_{NN}$	$\mathcal{M}_H$	$\mathcal{M}_R$
Base model (Sinkhorn inference)	**+1.98**	+2.02	+4.12	+2.02	+2.74	+4.22
Transformer-based encoder	+2.36	+1.48	+4.46	+2.24	+3.14	+5.92
CNOT-head	+2.14	+1.76	+3.92	+1.44	+3.18	+5.18
Probabilistic CNOT weights	+2.20	+1.68	+4.48	+1.64	+2.52	+5.44
Gumbel-Sinkhorn inference	**+1.56**	+1.70	+4.50	+1.40	+3.46	+6.20
CNOT self-attention	**+1.56**	+1.68	+4.32	+1.01	+3.00	+5.30
Entropy-guided search	**+1.49**	+1.69	+4.28	+0.91	+3.17	+5.09
Gumbel-Sinkhorn training	+2.21	+1.62	+4.25	+2.12	+2.85	+5.03
Wider encoder	**+1.52**	+1.66	+4.33	+1.04	+2.73	+5.41
CosineAnnealingWarmRestarts	**+1.45**	+1.66	+4.61	**−0.03**	+2.50	+5.19
Curriculum training	+1.70	+1.56	+4.50	+1.60	+2.85	+6.58

[1] The gap in C_{cx} to optimum (in gates)
[2] The gap in C_{dep} to optimum (in gates)
[3] Variants of the model described in Section 3.1

Table 1: Performance experiments vs. optimum in the 4-qubit benchmark

n (qubits)	ΔC_{cx} (in gates)			ΔC_{cx} (in %)		
	$\mathcal{M}_{NN}$	$\mathcal{M}_H$	$\mathcal{M}_M$ [1]	$\mathcal{M}_{NN}$	$\mathcal{M}_H$	$\mathcal{M}_M$
4	+1.52	+1.70	**+0.98**	+26.6	+29.7	+17.1
5	+3.20	+3.07	**+2.28**	+33.9	+32.6	+24.2

[1] The mixed model of $\mathcal{M}_{NN}$ and $\mathcal{M}_H$

Table 2: Performance of the mixed model (cumulative average in Appendix C)

3.1.5 Declaration of issues

Several fundamental issues in inference emerged when writing this report.

(1) We unconsciously regarded each $\hat{T}_i$ as a prediction of $\hat{p}_n$ and applied the Hungarian algorithm on each $\hat{T}_i$ to extract candidates of $\hat{p}_n$. Note that even T_n does not equivalently represent the cumulative effect of p_n, not to mention any intermediate T_i. This mismatch undermines the theoretical correctness of our model. However, the fact that it even outperformed the model $\mathcal{M}_H$ remains attractive. A possible explanation is that the small size ($(4!)^2 = 576$) of $\mathcal{P}_4$ might have constrained any significant discrepancy from the optimum, which is weakened by the observation that the model $\mathcal{M}_R$ already behaved incredibly far in the 4-qubit benchmark.

(2) We mistakenly implemented the model to predict $\hat{C}^*_{\text{cx}}$ for each T_i of the input circuit $\{T_i\}_{i=1}^n$, resulting in an n-long sequence of $\hat{C}^*_{\text{cx}}$. Instead, for the input circuit $\{T_i\}_{i=1}^n$, the prediction on $\hat{C}^*_{\text{cx}}$ should be a scalar. Hence, the CNOT loss should be an L1 regularization rather than MSE, and the CNOT weights in inference are invalid.

Fixing the first issue is more challenging. Though we can decompose $\{\hat{T}_i\}_{i=1}^n$ to extract predicted permutation matrices $\{\hat{P}_i\}_{i=1}^n$, we can not apply the Hungarian algorithm on each $\hat{P}_i$ to extract $(\hat{\sigma}(i), \hat{\tau}(i))$. We can try extracting the most likely row swap, but severe error propagation deserves circumspection. Nonetheless, these issues might suggest that previous attempts for model enhancement might have worked in a theoretically correct model. Due to time constraints, we leave corrections for future work.

3.2 Reinforcement learning

3.2.1 Motivation

Initial experiments were conducted on a fully connected NN, fed with separately encoded training data ($[X_s, Z_s]$ for stabilizers and $[X_d, Z_d]$ for destabilizers) and flattened target labels $\mathcal{Y}_n^*$. While the training loss steadily decreased, the validation loss converged to a high plateau (Figure 7, Appendix D), suggesting overfitting. We tested a wide range of model configurations, including varying the depth and dimension of hidden layers, tuning learning rates, setting up schedulers and early stopping, and increasing epochs up to 20,000. However, validation performance stagnated.

The structural sparsity of $\mathcal{Y}_n^*$ (Tables 3, 4, Appendix D) might partially explain. Over 93% zero entries globally induce high imbalance in $\mathcal{Y}_n^*$ and difficulty for the model to learn meaningful distinctions among pivot choices, which further incentivizes the model to converge toward trivial predictions (zero-valued $\hat{\mathcal{Y}}_n^*$). This minimized MSE over training data but failed to provide values for pivot selection since zero-valued $\hat{\mathcal{Y}}_n^*$ equalizes all pivot choices.

Fully connected NN even underperformed compared to the model $\mathcal{M}_R$ due to the lack of variances in target labels $\mathcal{Y}_n^*$. We thus transited to alternatives not relying on weak supervision, such as reinforcement learning.

3.2.2 Agent and environment architecture

Pivot selection ($p_n \in \mathcal{P}_n^*$) is treated as a sequential decision-making problem. We adopted a Deep Q-Network (DQN) agent to interact with a custom environment to reduce Clifford tableaux while minimizing C_{cx}. Specifically, it was an OpenAI Gym-style environment, `CliffordTableauEnv`, which simulates circuit synthesis from a Clifford tableau. At i-th step, the agent selects a pivot $(\sigma(i), \tau(i))$, triggering `steiner_reduce_column` for a reduction. The goal is to synthesize the identity using the fewest CNOT gates.

Our best-performing agent is based on Kremer's architecture [2], the setup of which (Appendix E) optimizes to minimize C_{cx} with auxiliary penalties on unnecessary CNOT gates and additional smaller penalties on S gates and H gates. We adopted curriculum learning to improve training efficiency: circuits were sampled with 5–21 gates analogous to lower-higher complexity, and complexity increased (so the curriculum window increased) as training progressed over 200,000 episodes.

We fine-tuned the model with constrained exploration for stable learning and updates

around high-performing behaviors using RL and supervised variants. The learned policy outperformed the model $\mathcal{M}_R$ with base training and further improved with fine-tuning but remained 30–40% above the optimum on average. It also underperformed compared to the model $\mathcal{M}_H$.

3.2.3 Factorized CNN

Our agent is a factorized DQN with a convolutional backbone. The $8 \times n \times n$ input tensor encodes $[X_s, Z_s]$, adjacency matrix, pivot mask, CNOT channel, and auxiliary coordinate channels. The CNN contains three residual blocks and fully connected layers that produce control and target qubit embeddings. They are combined via an outer product to yield the Q-value matrix $Q(i, j)$ over all valid pivot actions.

Curriculum learning started with 5-gate circuits and gradually increased to 21-gate. The staged exposure helps the agent acquire useful behaviors before tackling more complex synthesis tasks [2]. During base training, exploration combined ϵ-greedy decay with top-k softmax sampling to focus on the highest-scoring actions. A standard replay buffer stores transitions, and an archive tracks high-performing episodes. Fine-tuning prioritizes archived episodes for stability and refinement.

Rewards penalize gates (e.g. -1 per CNOT) and grant large final rewards based on the total C_{cx}. The agent used smooth L1 (Huber) loss and introduced optional reward-weighted loss scaling in fine-tuning to ensure that transitions with higher rewards contribute more strongly to weight updates, further coupling the optimization objective with the final synthesis quality:

$$\mathcal{L}_{\text{scaled}} = \mathcal{L} \times \left(1 + \alpha \cdot \frac{\text{reward}}{R_{\text{final}}}\right).$$

We tested (semi-)supervised reward conditioning on C^*_{cx} with auxiliary loss heads in CNN, but neither significantly improved the performance. The following supervised reward in fine-tuning yielded slight performance boosts at the cost of higher computational demand:

$$R = \begin{cases} R_{\text{final}}, & \text{if } \hat{C}^*_{\text{cx}} \leq C^*_{\text{cx}} \\ 0.5R_{\text{final}}, & \text{if } \hat{C}^*_{\text{cx}} \leq C^*_{\text{cx}} + 1 \\ 0.25R_{\text{final}}, & \text{if } \hat{C}^*_{\text{cx}} \leq C^*_{\text{cx}} + 2 \\ R_{\text{final}} \cdot \left(e^{-0.2(\hat{C}^*_{\text{cx}} - C^*_{\text{cx}})} - 1\right), & \text{otherwise.} \end{cases}$$

This reward encourages near-optimal behavior but limits the batch size and episode count due to the high cost of computing C^*_{cx} per environment.

We experimented with two auxiliary prediction heads alongside the main Q-network for additional supervision. The CNOT classification head is a binary classifier that predicts whether the final $\hat{C}^*_{\text{cx}}$ will fall within the tolerance (for example, $\leq C^*_{\text{cx}} + 1$). It acts as a coarse indicator of near-optimality. The CNOT regression head is a continuous regressor that predicts $\hat{C}_{\text{cx}}$ at episode end, encouraging the network to encode long-term outcome information. Both heads share the CNN backbone with the Q-network and

are trained via cross-entropy (for classification) and MSE (for regression). These auxiliary losses were optionally weighted and integrated into the main Q-value loss during optimization. Experiments proved that these two heads led to no gain in performance.

3.2.4 Performance analysis

Recall that our best RL-based model underperformed compared to the model $\mathcal{M}_H$ and stayed 30–40% higher in C_{cx} than the optimum. We attribute this to three primary issues:

1. **Instability between underfitting and overfitting:** Base training failed to learn effective pivot selection strategies, performing slightly better than the model $\mathcal{M}_R$. Fine-tuning with replayed near-optimal episodes initially reduced C_{cx} but overfitted to memorized trajectories. Lowering the archive replay ratio reintroduced exploration but degraded performance, highlighting the difficulty in balancing generalization and close-to-optimum learning.

2. **Poor scaling with circuit complexity:** The agent performed reliably on trivial circuits (0–2 gates, Table 5, Appendix F), but generalization deteriorated on more complex cases. For circuits with 10 gates, C_{cx} varied wildly even after fine-tuning, sometimes exceeding 12 (Figure 7, Appendix F). We failed to constrain the solution quality as circuit complexity grows.

3. **Reward and loss sensitivity:** Reward shaping based on C^*_{cx} encouraged short-term convergence but was fragile upon reward computation inaccuracies or sparse high-reward trajectories. The imperfect scaling in the loss function led to unstable gradients and plateaus. The reward-weighted loss helped in the early stages but did not generalize well across curriculum stages or circuit complexity.

3.2.5 Alternative agent

We experimented with a dueling DQN [5] agent using a CNN encoder and a factorized Q-head tailored to the pivot action space $\mathcal{P}_n$ to explore architectural improvements. This agent also incorporated an auxiliary head to estimate state optimality. While it offered improvements in C_{cx} over the model $\mathcal{M}_R$, it consistently underperformed compared to the model $\mathcal{M}_H$. Full architectural and training details are available in Appendix G.

3.3 Recurrent neural network

Iteratively applying Clifford gates during circuit synthesis is inherently sequential, with each step reducing the tableau based on the current pivot choice [6]. This motivated experimentation with RNNs, particularly Long Short-Term Memory (LSTM) models [1], which can learn temporal dependencies. A CNN was used to extract spatial features from each tableau to incorporate structural awareness, which was then passed to an LSTM for temporal processing.

Two modeling tasks were explored:

- **Autoregressive tableau prediction:** Given a subsequence of tableau states $R[i : i + s - 1]$ during circuit synthesis, predict the next state $R[i + s]$. Training data were generated by brute force and sliced into overlapping subsequences of length s. The model showed limited success—sometimes repeating matrices or jumping to the final state too early.
- **Pivot prediction:** Using the whole sequence of tableau states $R[0 : n]$ during circuit synthesis, a similar CNN-LSTM model was trained to predict the correct pivot at each step. The model achieved good training loss, indicating successful learning when the ground-truth tableau state evolution was available.

Experiment results (Appendix H) suggest that while LSTM-based modeling for pivot selection holds promise, improved architectural design and loss functions are needed for viable sequence generation.

4 Justification for Grade 5

Despite a slow start during the first two months due to the project's abstract nature and research-heavy focus, we ultimately achieved the client's goal of outperforming the models $\mathcal{M}_R$ and $\mathcal{M}_H$ [6] on the 4-qubit benchmark. The absence of publication-ready results within four months does not diminish this success; rather, it reflects our persistent, collaborative efforts in exploration, especially between the midterm and final presentations.

Each member contributed strategically based on their expertise in presentations, theory, or implementations. To maintain momentum over the four months, we enforced efficient collaboration through weekly focused meetings and consistent real-time progress tracking, even on weekends.

While time constraints led to minor conceptual inaccuracies during three presentations, continuous peer feedback ensured steady technical and strategic growth.

References

[1] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

[2] David Kremer, Victor Villar, Hanhee Paik, Ivan Duran, Ismael Faro, and Juan Cruz-Benito. Practical and efficient quantum circuit synthesis and transpiling with reinforcement learning. *arXiv preprint arXiv:2405.13196*, 2024.

[3] Animesh Sinha, Utkarsh Azad, and Harjinder Singh. Qubit routing using graph neural network aided monte carlo tree search. In *Proceedings of the AAAI conference on artificial intelligence*, volume 36, pages 9935–9943, 2022.

[4] Arianne Meijer van de Griend and Sarah Meng Li. Dynamic qubit routing with cnot circuit synthesis for quantum compilation. *Electronic Proceedings in Theoretical Computer Science, EPTCS*, 394:363–399, 2023.

[5] Ziyu Wang, Nando de Freitas, and Marc Lanctot. Dueling network architectures for deep reinforcement learning. *CoRR*, abs/1511.06581, 2015.

[6] David Winderl, Qunsheng Huang, Arianne Meijer-van de Griend, and Richie Yeung. Architecture-aware synthesis of stabilizer circuits from clifford tableaus. *arXiv preprint arXiv:2309.08972*, 2023.

A Preliminaries

A.1 Qubits and quantum states

The standard basis states of 1 qubit are

$$|0\rangle = [1,0]^\top,\ |1\rangle = [0,1]^\top.$$

The standard basis states of 2 qubits are

$$|00\rangle = [1,0,0,0]^\top,\ |01\rangle = [0,1,0,0]^\top,\ |10\rangle = [0,0,1,0]^\top,\ |11\rangle = [0,0,0,1]^\top.$$

A 1-qubit quantum state can be generally denoted by

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle = [\alpha,\beta]^\top,$$

where $\alpha, \beta \in \mathbb{C}$ (complex numbers), $(\alpha,\beta) \neq (0,0)$, $|\alpha|^2 + |\beta|^2 = 1$ and α, β are called amplitude. A 2-qubit quantum state can be generally denoted by

$$|\psi\rangle = \alpha|00\rangle + \beta|01\rangle + \gamma|10\rangle + \lambda|11\rangle = [\alpha,\beta,\gamma,\lambda]^\top,$$

where $\alpha, \beta, \gamma, \lambda \in \mathbb{C}$ (complex numbers), $(\alpha,\beta,\gamma,\lambda) \neq (0,0,0,0)$, $|\alpha|^2+|\beta|^2+|\gamma|^2+|\lambda|^2 = 1$ and $\alpha, \beta, \gamma, \lambda$ are called amplitude.

A.2 SWAP = 3 CNOT gates

SWAP is a 2-qubit gate that swaps the states of two qubits. For the standard basis states,

$$\text{SWAP}|01\rangle = |10\rangle,\ \text{SWAP}|10\rangle = |01\rangle.$$

In the form of matrix representation, SWAP for a general 2-qubit state is denoted by

$$\text{SWAP}|\psi\rangle = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} [\alpha,\beta,\gamma,\lambda]^\top = [\alpha,\gamma,\beta,\lambda]^\top.$$

CNOT is another 2-qubit gate, where the NOT gate applies to the target qubit according to the state of the controlled qubit. For the standard basis states,

$$\text{CNOT}|00\rangle = |00\rangle,\ \text{CNOT}|01\rangle = |01\rangle,\ \text{CNOT}|10\rangle = |11\rangle,\ \text{CNOT}|11\rangle = |10\rangle,$$

or,

$$\text{CNOT}|00\rangle = |00\rangle,\ \text{CNOT}|01\rangle = |11\rangle,\ \text{CNOT}|10\rangle = |10\rangle,\ \text{CNOT}|11\rangle = |01\rangle.$$

In the form of matrix representation, CNOT for a general 2-qubit state is denoted by

$$\mathrm{CNOT}|\psi\rangle = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} [\alpha, \beta, \gamma, \lambda]^\top = [\alpha, \beta, \lambda, \gamma]^\top,$$

or

$$\mathrm{CNOT}|\psi\rangle = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} [\alpha, \beta, \gamma, \lambda]^\top = [\alpha, \lambda, \gamma, \beta]^\top.$$

We can see that

$$\mathrm{SWAP} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

That is, a SWAP gate can be decomposed into 3 CNOT gates.

B SWAP insertions solve qubit routing

Consider the 3×3 grid architecture in Figure 1a with edges only between neighboring qubits and the 9-qubit circuit in Figure 2a. We ignore these two successive CNOT gates between qubits q_3 and q_4 in our discussion because they cancel out:

$$\text{CNOT} \circ \text{CNOT}|q_3, q_4\rangle = |q_3, q_4 \oplus q_3 \oplus q_3\rangle = |q_3, q_4\rangle.$$

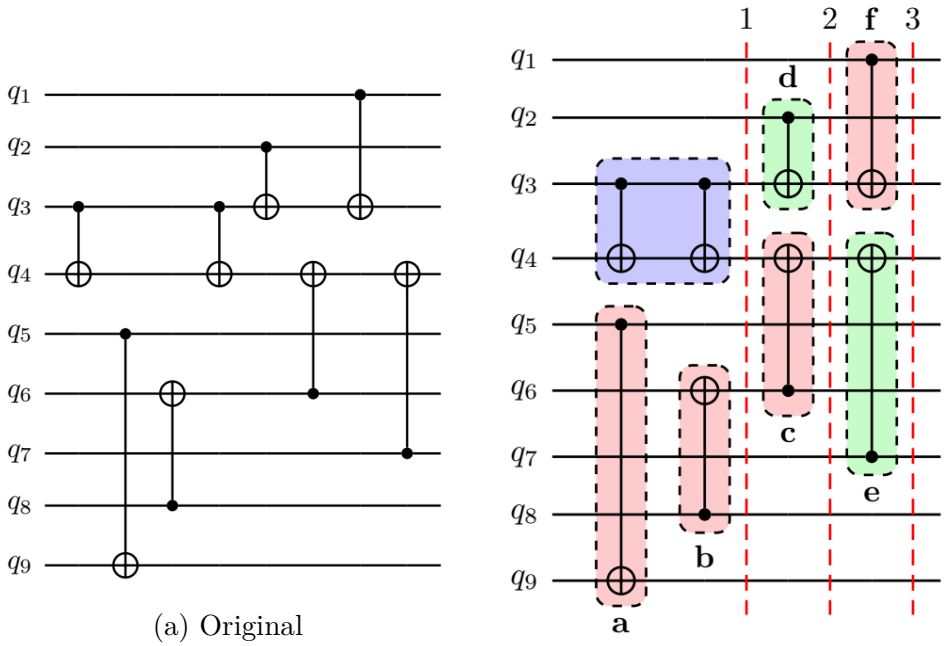
1	2	3
4	5	6
7	8	9

(a) Original

1	3	2
7	5	9
4	6	8

(b) After 4 SWAP

Figure 1: 3×3 grid architecture



(a) Original

(b) Decomposed

Figure 2: 9-qubit circuit [3]

The operations $a : q_5 \mapsto q_9$, $b : q_8 \mapsto q_6$, $c : q_6 \mapsto q_4$, and $f : q_1 \mapsto q_3$ do not comply with the architecture in Figure 1a because these qubits involved in a, b, c, or f are not adjacent. Thus, we apply 4 SWAP between (q_6, q_9), (q_4, q_7), (q_2, q_3), and (q_6, q_8) in order and achieve the architecture in Figure 1b. Originally, the 9-qubit circuit in Figure 2a can be decomposed into 3 slices (Figure 2b), within each of which the operations can be executed parallelly. The circuit depth is thus 3. However, after inserting 4 SWAP

gates, the transformed circuit can only be decomposed into 5 slices, as shown in Figure 3, resulting in a circuit depth of 5.

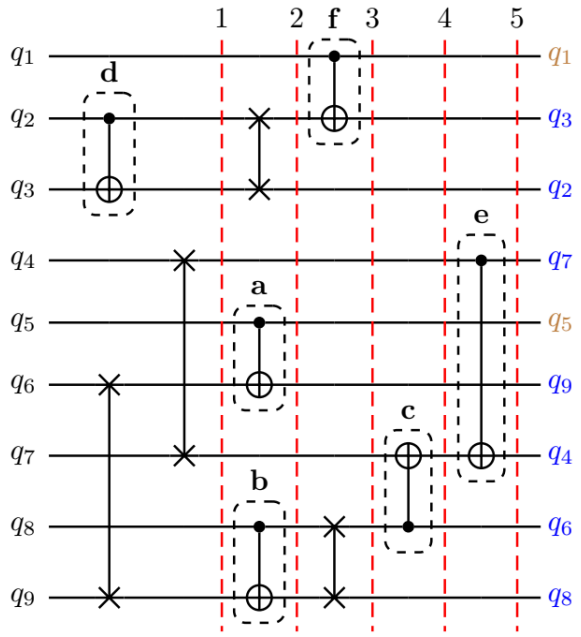


Figure 3: 9-qubit circuit with 4 SWAP [3]

C Performance of the mixed model

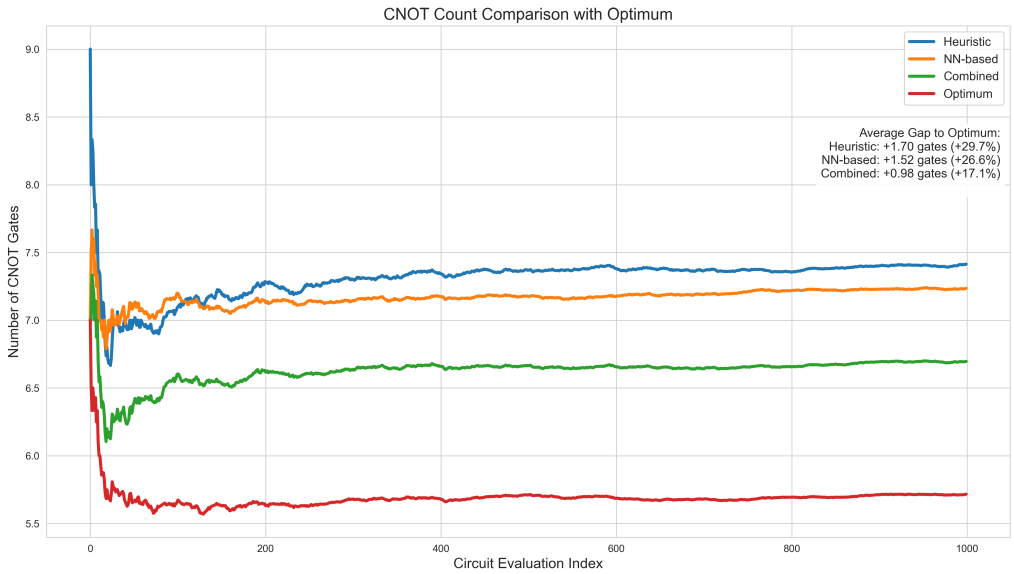


Figure 4: Cumulative average of the 4-qubit benchmark in Table 2

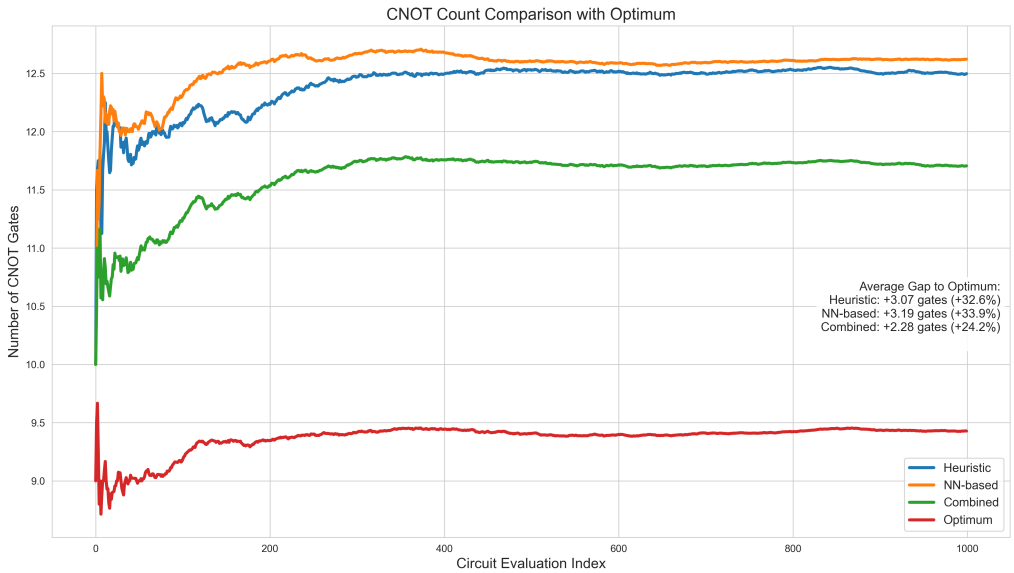


Figure 5: Cumulative average of the 5-qubit benchmark in Table 2

D Fully connected NN with sparse target labels

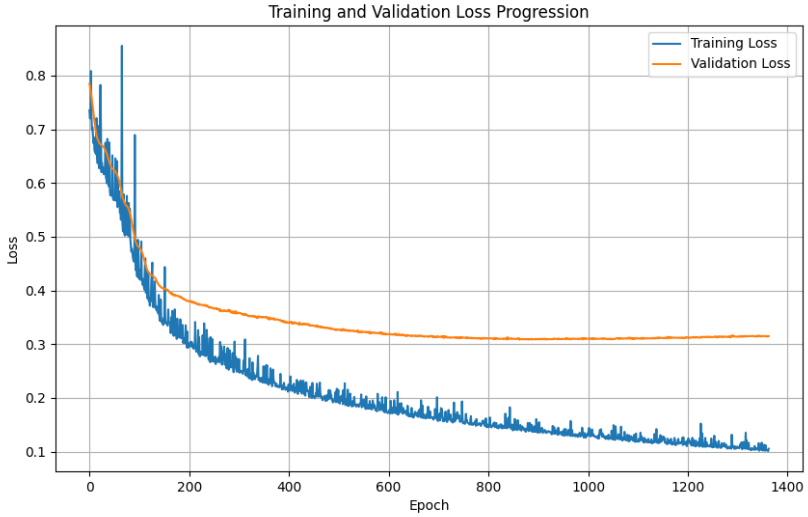


Figure 6: Training/validation loss of fully connected NN (3 64-neuron hidden layers)

Value	**Occurrences**
0	7076
1	185
2	275
3	122
4	40
5	43
6	21
7	7
8	2
9	4
10	1

Table 3: Global value distribution in $\mathcal{Y}_n^*$

Condition	**Count (out of 486)**
All-zero matrices	329 (67.70%)
Matrices with $\geq 50\%$ non-zero entries	17 (3.50%)
Matrices with at least 1 non-zero	157 (32.30%)

Table 4: Matrix-level analysis of sparse $\mathcal{Y}_n^*$

E Diagram of the RL-based model

The following diagram summarizes the full training and interaction loop of our RL system during fine-tuning.

```
[Random Circuit Generation]
    --> random_hscx_circuit(n_qubits, nr_gates)

[Environment Reset]
    |-- Converts to Tableau
    |-- Inverts Tableau
    |-- Computes Optimal CX Count (get_best_cnots)

[Agent Action Selection]
    --> act(state, allowed_rows, allowed_cols, explore=True)

[Environment Step]
    |-- Applies pivot to tableau
    |-- Records gate usage (H, S, CX)
    |-- Updates reward:
    |     --> _compute_supervised_reward(found, optimal)
    --> Marks done when all qubits reduced

[Experience Storage]
    |-- remember(...)
    --> remember_episode(done)
          |-- Adds to full memory
          |-- Adds to archive if:
                * CX   threshold × optimal
                * archive size < limit

[Training: agent.replay()]
    |-- Sample batch:
    |     |-- archive_batch   archive_ratio
    |     --> memory_batch   (1 - archive_ratio)
    |-- Compute target Q:
    |     --> y = r +   × max(Q(s ))
    |-- Optional loss scaling:
    |     --> loss *= (1 +   × reward / R_max)
    --> Backpropagation:
          - optimizer.step()
          - gradient clipping
```

F Performance of the RL-based model

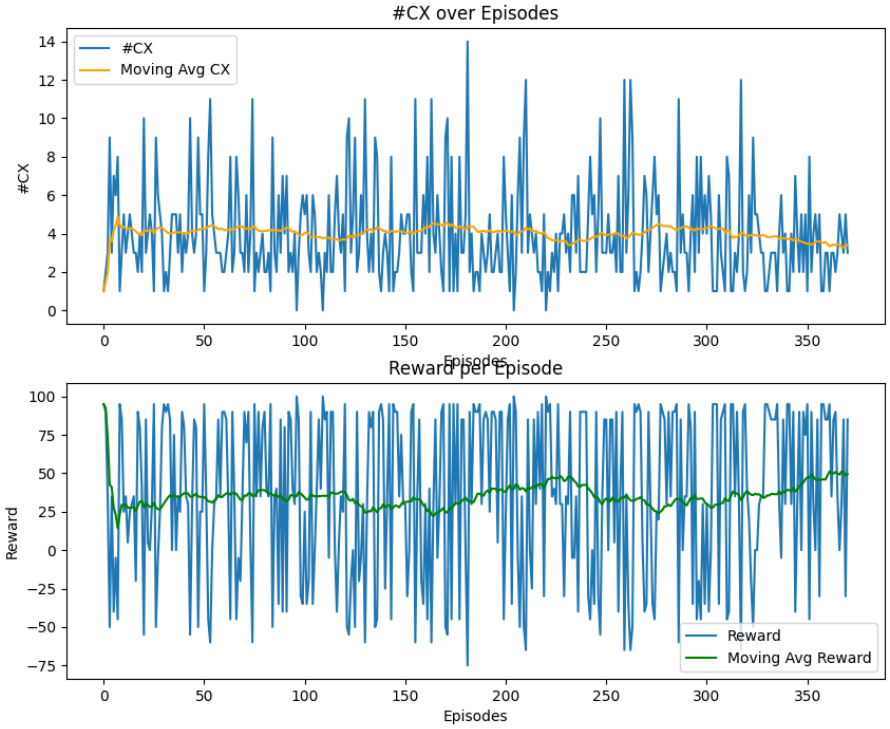


Figure 7: Supervised fine-tuning of RL agent on 10-gate circuits, displaying convergence towards near-optimum moving average CNOT count. Note the variance in CNOT selections and occasional high-CNOT spikes.

C^*_{cx} \ $\hat{C}^*_{\mathrm{cx}}$	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
0	40%	0%	0%	60%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
1	0%	79%	0%	0%	20%	0%	0%	0%	0%	0%	2%	0%	0%	0%	0%
2	0%	0%	64%	14%	0%	6%	0%	0%	7%	9%	0%	0%	0%	0%	0%
3	0%	0%	0%	0%	0%	0%	26%	26%	0%	0%	0%	28%	12%	7%	1%
4	0%	0%	0%	0%	0%	0%	0%	0%	100%	0%	0%	0%	0%	0%	0%
5	0%	0%	0%	0%	0%	100%	0%	0%	0%	0%	0%	0%	0%	0%	0%
6	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	100%	0%
7	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	100%	0%	0%	0%

Table 5: Confusion matrix ($\hat{C}^*_{\mathrm{cx}}$ by RL agent vs. C^*_{cx}) over 1000 evaluations. Diagonal entries, where $\hat{C}^*_{\mathrm{cx}} = C^*_{\mathrm{cx}}$, are shaded.

G Dueling DQN agent

The following outlines the architecture and learning procedures of our investigated alternative dueling DQN agent.

- **Convolutional Backbone:** The agent processes a $4 \times n \times n$ tensor (with channels for Pauli-X, Pauli-Z, adjacency matrix, pivot mask) through a lean CNN: one 3×3 convolution with 64 filters $\rightarrow$ ReLU $\rightarrow$ two 64-channel residual blocks $\rightarrow$ adaptive average pooling $\rightarrow$ flatten $\rightarrow$ Linear(256) $\rightarrow$ ReLU. The final output is a compact 256-dim feature vector shared by all heads.
- **Dueling & Factorized Q-Heads:** From the shared 256-dim features, three parallel streams are computed:
  - a value head (Linear(256, 1)) estimating $V(s)$;
  - two advantage heads (each Linear(256, n)) for the controlled qubit (A_c) and the target qubit (A_t);
  - an auxiliary head (Linear(256, 1) $\rightarrow$ sigmoid) predicting state optimality.

  The full $Q(i, j)$ matrix is reconstructed as

$$Q(s, i, j) = V(s) \; + \; \Big[A_c(i)\, A_t(j) \; - \; \text{mean}(A_c \otimes A_t) \Big].$$

- **N-Step Returns & Replay:** Transitions are gathered into a 3-step buffer before storage (maximum length 20,000) with their terminal auxiliary label. During learning, random minibatches of these 3-step tuples are sampled, and a frozen target network (periodically synced) supplies the bootstrap $\max_{i,j} Q_{\text{target}}(s', i, j)$.
- **Pure ϵ-Greedy Exploration:** With probability ϵ, the agent picks a random valid pivot; otherwise, it greedily selects the (i, j) that maximizes the dueling Q-matrix. ϵ is decayed multiplicatively at each training step to a configurable minimum.
- **Auxiliary Supervision & Annealing:** On episode termination, the final C_{cx} is compared against an optimal estimate $\hat{C}^*_{\text{cx}}$ to produce a continuous label in $[0, 1]$. The auxiliary head is trained via BCE on these labels, with its loss weight $\alpha(t)$ linearly annealed from an initial to a final value over a set number of steps, emphasizing auxiliary shaping early and pure Q-learning later.
- **Loss Composition:** Each update minimizes

$$\mathcal{L} = \underbrace{\text{Huber}\big(Q(s, a) \; - \; [r + \gamma \, \max Q_{\text{target}}]\big)}_{\mathcal{L}_Q} + \alpha(t) \; \underbrace{\text{BCE}\big(\hat{p}_{\text{opt}}(s), \, y_{\text{aux}}\big)}_{\mathcal{L}_{\text{aux}}}.$$

- **Initialization & Regularization:** The model relies on PyTorch's default initializers and the stability of residual connections to focus capacity on the dueling and auxiliary objectives.

H CNN + LSTM outcomes

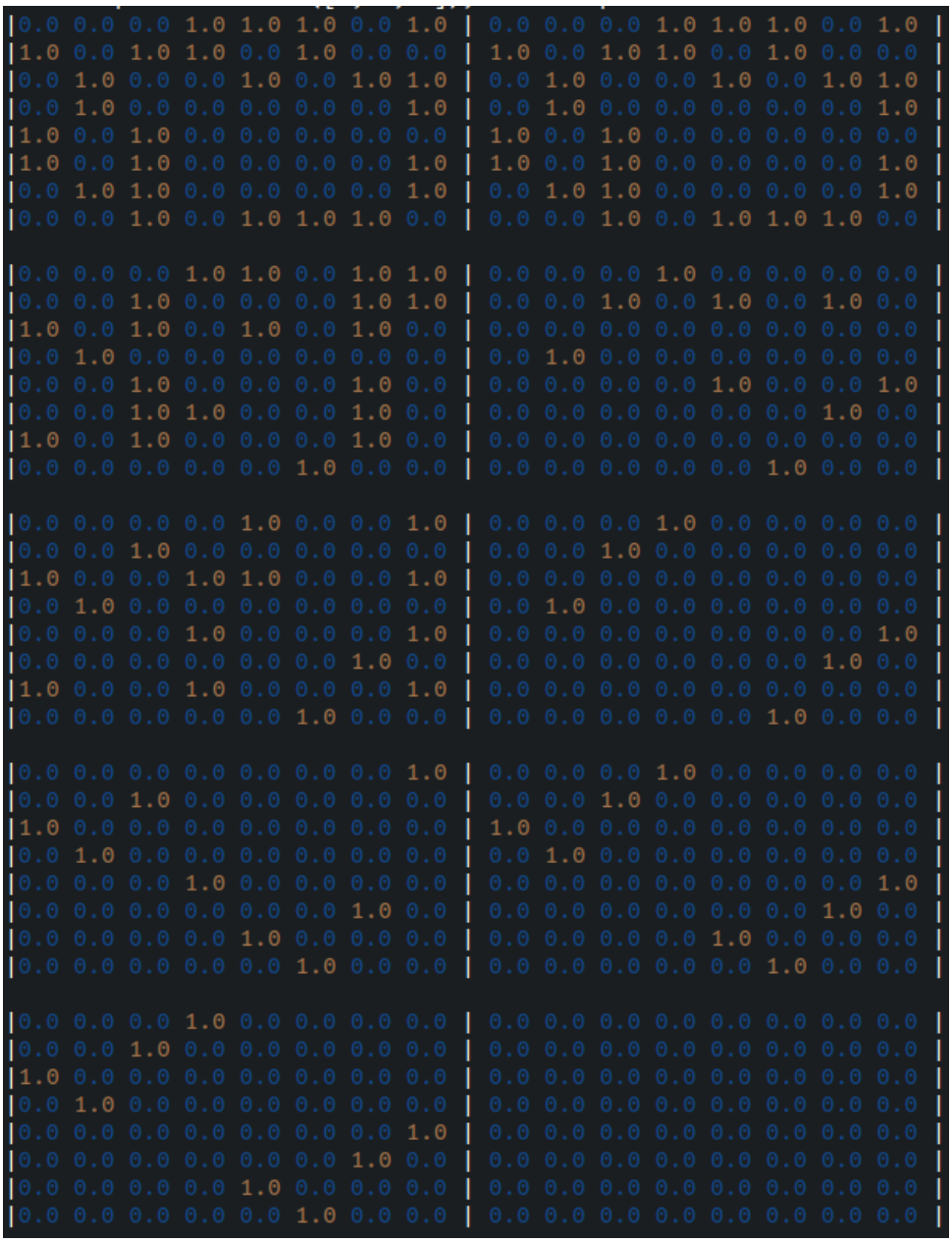


Figure 8: CNN + LSTM trained with sparse matrix sequences. Left: optimal sequence; Right: predicted sequence. Note there is much information loss.